Product: SheSecure

Category: Women’s personal safety and emergency assistance

Platform: Native Android application

Primary technology: Java, XML, Android Studio

Backend: Supabase Auth and PostgreSQL

Version: MVP 1.0

Core concept: A user can register, manage trusted emergency contacts, and trigger an emergency alert manually through an SOS button or automatically through a configured shake gesture. The application attempts to obtain the user's current location and send an emergency SMS to the user's saved contacts.

1. Product vision

SheSecure aims to provide a simple, dependable, and accessible personal safety tool that enables users to alert trusted people when they feel unsafe or need urgent assistance.

The app should minimize the steps required to trigger an alert, make emergency contacts easy to manage, and offer a polished, professional experience.

SheSecure is an assistance tool, not a replacement for emergency services. Delivery of SMS messages, availability of GPS location, and successful alert transmission cannot be guaranteed.

2. Problem statement

During an emergency, a person may not have enough time to unlock their phone, open a messaging application, find a trusted contact, type a message, and share their location.

SheSecure addresses this problem through:

- A dedicated, easily accessible SOS button.
- Configurable shake-to-trigger functionality.

A predefined list of trusted emergency contacts.

-
-

Automatic preparation of an emergency message containing location details when available.

-
-

A persistent account and cloud-backed contact management system.

-

3. Target users

-

Women who want a convenient way to notify trusted contacts.

-
-

Students, commuters, working professionals, and people travelling alone.

-
-

Users who need a straightforward personal safety application.

-

The application should also support users who are not technically experienced.

4. Product goals

1.

Provide reliable, understandable emergency-triggering behavior.

2.
3.

Allow users to manage emergency contacts at any time.

4.
5.

Keep user accounts and contact data synchronized through Supabase.

6.
7.

Preserve login sessions across application restarts.

8.
9.

Keep emergency actions as independent as practical from internet availability.

10.
11.

Deliver a consistent, professional Android UI.

12.
13.

Handle permissions, failures, and accessibility explicitly.

14.

Non-goals for MVP 1.0

-

Live tracking of users by other people.

-
-

An emergency-response control centre.

-
-

AI-based danger detection.

-
-

A public social network or community feed.

-
-

Payment systems or advertisements.

-
-

Guaranteed SMS delivery or guaranteed background operation.

-

These may be evaluated later without complicating the initial product.

5. Functional requirements

FR-01: Onboarding and authentication

-

Provide welcome, signup, login, and logout screens.

-
-

Support email/password authentication through Supabase Auth.

-
-

Include input validation, password visibility, loading indicators, and clear error messages.

-
-

Persist authentication sessions securely and restore the correct screen when the app reopens.

-
-

Provide password reset functionality.

-
-

Prevent unauthenticated users from accessing protected account screens.

-
-

Do not require users to sign in again every time they launch the app unless their session has expired or been revoked.

-

FR-02: User profile

-

Allow users to view and update their display name.

-
-

Show account information and logout controls.

-
-

Store the profile against the authenticated Supabase user ID.

-
-

Avoid collecting unnecessary personal information.

-

FR-03: Emergency contacts

-

Allow users to add a contact by entering a name and phone number.

-
-

Validate phone-number input and support the appropriate country code.

-
-

Allow contacts to be edited and deleted.

-
-

Display the saved contacts in a clearly organized list.

-
-

Set a configurable limit, initially five contacts.

-
-

Prevent duplicate entries where appropriate.

-
-

Provide helpful empty states when no contacts have been added.

-
-

Synchronize contact changes with Supabase when connectivity is available.

-
-

Make the locally available emergency contacts accessible to the emergency workflow without waiting for a cloud request.

-

The app should explain that a valid phone number and SMS-capable service are required for text-message alerts.

FR-04: Manual SOS

-

Display a prominent SOS control on the main dashboard.

-
-

Require an intentional activation gesture, such as pressing and holding the button, to reduce accidental triggers.

-
-

Show a clear confirmation or countdown that the user can cancel.

-
-

On confirmation, initiate the emergency workflow.

-
-

Obtain a recent location if permission and device services allow.

-
-

Generate a concise emergency message with a location link when available.

-
-

Attempt to send the alert to each saved emergency contact.

-
-

Report success, partial success, or failure accurately where the platform exposes that information.

-

Do not show a successful delivery status merely because an SMS send operation was initiated.

FR-05: Shake-to-trigger

-

Provide a setting to enable or disable shake detection.

-
-

Allow the sensitivity threshold to be configured through a simple setting.

-
-

Detect deliberate shaking through Android sensor APIs.

-
-

Use thresholding, cooldowns, and duplicate-trigger protection to reduce accidental activations.

-
-

Route a valid shake event through the same emergency workflow as manual SOS.

-
-

Provide a way to cancel an accidental trigger.

-
-

Explain when shake detection is active and its limitations.

-

Platform constraint: Shake detection is straightforward while the app is active. Reliable detection when the app is in the background or the phone is locked requires separate Android lifecycle, sensor, foreground-service, battery, and operating-system restriction analysis. It must not be advertised as universally available until validated on supported devices.

FR-06: Location

-

Request location permission only when it is relevant to the feature.

-
-

Explain why location is needed before requesting access.

-
-

Obtain the best available recent location within a defined timeout.

-
-

Include a Google Maps-compatible location link when coordinates are available.

-
-

If location retrieval fails, still attempt the emergency message without coordinates.

-
-

Display clear feedback if location services are disabled or permission is denied.

-
-

Avoid continuous location collection in MVP 1.0.

-

FR-07: SMS alert handling

The alert should contain:

-

A short emergency notification.

-
-

The user's name or chosen identifier, where available.

-
-

Coordinates or a map link when available.

-
-

A brief instruction to contact the user or seek appropriate help.

-

Implementation must account for Android SMS permissions, device support, SIM/service availability, operating-system restrictions, and distribution policies. SMS sending must be tested on physical devices.

Where direct SMS sending is unavailable or restricted, provide an appropriate fallback, such as opening the system SMS composer with the prepared message. Clearly communicate that this fallback may require user interaction.

FR-08: Dashboard

The main dashboard should provide:

-

A personalized greeting.

-
-

A clearly visible SOS action.

-
-

A compact safety-status area.

-
-

Emergency-contact summary.

-
-

Quick access to manage contacts.

-
-

Shake detection status and settings.

-
-

Access to profile and app settings.

-

Avoid decorative statistics or safety scores that have no meaningful data behind them.

FR-09: Settings

Provide settings for:

-

Shake detection enable/disable.

-
-

Shake sensitivity, if supported.

-
-

Emergency contact management.

-
-

Permission status and instructions.

-
-

Profile and logout.

-
-

Privacy policy and safety disclaimer.

-
-

App version and basic troubleshooting.

-

FR-10: Error and offline handling

-

Show useful loading, empty, error, and retry states.

-
-

Keep locally available contacts usable during temporary network outages.

-
-

Queue or synchronize ordinary data changes when appropriate.

-
-

Do not delay the emergency workflow while waiting for Supabase.

-
-

Clearly distinguish an attempted alert from a confirmed delivery.

-
-

Handle missing location, missing contacts, denied permissions, and SMS failures.

-

6. UI/UX and visual design requirements

The UI must look like a deliberately designed commercial mobile application, not an automatically assembled template.

Design direction

-

Calm, reassuring, professional, and discreet.

-
-

Strong visual hierarchy, especially on emergency screens.

-
-

Consistent typography and spacing.

-
-

Restrained use of color.

-
-

Clear iconography and meaningful labels.

-
-

Minimal decoration and no unnecessary animation.

-

Visual system

-

Use a controlled palette: deep navy or charcoal for structure, warm off-white for light surfaces, and a restrained red or coral for emergency actions.

-
-

Use a neutral, legible sans-serif typeface with a consistent type scale.

-
-

Use a spacing system based on consistent increments, such as 4dp and 8dp.

-
-

Use consistent corner radii, elevation, borders, and icon sizes.

-
-

Reserve the emergency color for important safety actions and warnings.

-
-

Design both light and dark themes only if they can be implemented consistently.

-

Anti-vibe-coding requirements

-

No random gradients or excessive glassmorphism.

-
-

No glowing effects, cursor-following effects, or decorative animations.

-
-

No excessive rounded cards around every element.

-
-

No emoji used as a substitute for a proper icon system.

-
-

No inconsistent fonts, spacing, colors, or button styles.

-
-

No invented statistics, fake map activity, fake online statuses, or decorative charts.

-
-

No placeholder buttons that appear functional but do nothing.

-
-

No generic dashboard sections added only to fill space.

-
-

No default system-looking screens mixed with a completely different custom design without a reason.

-

Every screen must have a clear purpose and a consistent visual language.

7. Required screens

Screen	Purpose
Splash / session restoration	Initialize app and determine the correct destination
Welcome / onboarding	Introduce the product and permissions when relevant
Signup	Create an account
Login	Authenticate an existing user
Password recovery	Start password reset
Main dashboard	Access SOS and essential safety controls
Emergency contacts	Add, edit, delete, and view trusted contacts
Add/edit contact	Enter and validate contact details
SOS countdown	Prevent accidental alert activation
Alert status	Display location and SMS outcomes accurately
Settings	Configure shake behavior and app preferences
Profile	View and edit basic account information
Permission guidance	Explain required permissions and how to enable them

Navigation should be simple and predictable. Emergency actions should not be buried in a menu.

8. Technical architecture

Android client

-

Java for application logic.

-
-

XML for native Android layouts.

-
-

Android SDK APIs for lifecycle, sensors, location, and SMS.

-
-

A clear separation between UI, business logic, data access, and platform services.

-

Backend

-

Supabase Auth for authentication.

-
-

PostgreSQL for user profiles and emergency contacts.

-
-

Row Level Security for user-level data access.

-
-

Supabase client integration compatible with the chosen Android/Java implementation.

-

Suggested package structure

```
com.shesecure
├── auth
├── dashboard
├── contacts
├── emergency
├── location
```

```
├──── sensors
├──── settings
├──── profile
├──── data
│    ├──── local
│    ├──── remote
│    └──── repository
├──── models
├──── utils
└──── ui
```

This is a logical organization; implementation should follow a consistent, buildable Android project structure rather than creating empty packages unnecessarily.

9. Database requirements

`profiles`

-

`id`: UUID linked to `auth.users.id`.

-
-

`display_name`: text.

-
-

`created_at`: timestamp.

-
-

`updated_at`: timestamp.

-

`emergency_contacts`

-

`id`: UUID primary key.

-
-

`user_id`: UUID linked to `auth.users.id`.

-

- `contact_name`: text.
- `phone_number`: text.
- `created_at`: timestamp.
- `updated_at`: timestamp.

Apply RLS policies so an authenticated user can read, insert, update, and delete only their own records. Configure appropriate foreign keys, validation, and indexes.

Never expose a Supabase service-role secret in client code. Public client configuration does not replace database security policies.

10. Privacy and security

- Collect only the information needed for the product's functionality.
- Request permissions in context and explain their purpose.
- Do not continuously track users without a separate, explicit feature and consent.
- Restrict access to each user's cloud data.

Do not log passwords, access tokens, or unnecessary sensitive information.

-
-

Keep secrets out of GitHub.

-
-

Provide a clear privacy policy describing contact and location use.

-
-

Treat emergency contacts and location data as sensitive information.

-
-

Provide a reasonable account and data deletion process.

-

11. Performance and reliability

-

The dashboard should load promptly on ordinary Android devices.

-
-

Authentication and contact operations should show clear progress feedback.

-
-

A network failure must not freeze the emergency interface.

-
-

Sensor processing should be lightweight and include debouncing.

-
-

Repeated triggers should not create uncontrolled duplicate SMS attempts.

-

-

Location requests should have a timeout.

-
-

The app should handle lifecycle changes, rotation where supported, and process recreation gracefully.

-

12. Testing and acceptance criteria

The MVP is not complete until these cases are tested.

-

New user can sign up and log in.

-
-

Existing user remains logged in after restarting the app.

-
-

Logout and password recovery behave correctly.

-
-

One user cannot access another user's contacts.

-
-

Contacts can be added, edited, deleted, and reloaded.

-
-

The dashboard handles an empty contact list.

-
-

Manual SOS follows the confirmation flow.

-
-

Shake detection works at the documented operating conditions.

-
-

Location is included when available and handled gracefully when unavailable.

-
-

SMS behavior is tested on physical devices.

-
-

Missing permissions and missing SMS service are handled correctly.

-
-

Network failure does not block the emergency workflow.

-
-

Repeated triggers do not send uncontrolled duplicate messages.

-
-

Loading, error, empty, success, and partial-failure states are designed.

-
-

All screens follow the same design system.

-
-

A clean build succeeds without unresolved dependencies or compilation errors.

-

-

No secrets are committed to the repository.

-

13. Delivery phases

1.

Foundation: Create a clean native Android project and establish the design system.

2.
3.

UI implementation: Build and review all main screens with realistic navigation.

4.
5.

Authentication: Configure Supabase Auth and session persistence.

6.
7.

Contact management: Implement cloud storage, RLS, and local caching.

8.
9.

Emergency features: Add manual SOS, location, SMS, and shake detection.

10.
11.

Reliability: Test offline behavior, permissions, cancellation, and duplicate protection.

12.
13.

Release preparation: Final UI review, signed build, privacy policy, and distribution checks.

14.

14. MVP completion definition

SheSecure 1.0 is complete when the core account, contact-management, and emergency-alert flows work on a real Android device; cloud data is properly secured; errors are handled transparently; and the UI is consistent, accessible, and professionally finished.

The product must never claim that a message or alert was delivered unless that outcome is actually known.